ENGINEERING

The Art of Reading Code: Inside Simpyll’s New Journal Experience

A hands-on tour of code highlighting, responsive tables, callouts, figures, disclosures, footnotes, and the details that make technical writing a pleasure to read.

David Becker · October 22, 2026 · 5 min read

Simpyll · https://simpyll.com/blog/the-art-of-reading-code

A field guide to the details that make technical writing worth exploring.

A good developer blog tells you what someone built. A great developer blog makes you want to explore **how** they built it.

There is a difference between reading a wall of documentation and exploring a thoughtfully designed technical article. One delivers information. The other helps you navigate, understand, and use it.

This post is also a live demonstration. Try the copy buttons, open the disclosures, follow a heading link, and read the comparison table on your phone.

01. The little things matter

Great reading experiences are built from small decisions: how headings breathe, how code is presented, how tables behave on narrow screens, and whether a link takes you where you expected.

> ⓘ **Information: A design principle**
>
> The interface should make the content easier to understand, not compete with it for attention.

The goal is calm while reading, responsive while exploring, and predictable while navigating.

02. Code that deserves to be read

Code is not decoration. Often, it is the most important part of a technical article.

typescript

```typescript
function calculateReadingTime(
  text: string,
  wordsPerMinute = 225
): number {
  const words = text.trim().split(/\s+/).filter(Boolean).length
  return Math.max(1, Math.ceil(words / wordsPerMinute))
}

console.log(calculateReadingTime("Build something useful."))
```

Notice the language label, highlighting, spacing, and **copy** control. The copy button should give you the original code, not the toolbar or formatting artifacts. Highlighting happens at build time, so the article remains readable before client-side JavaScript runs.

The same idea in Python

python

```python
import math

def reading_time(text: str, words_per_minute: int = 225) -> int:
    words = len(text.split())
    return max(1, math.ceil(words / words_per_minute))

print(reading_time("Build something useful."))
```

And a shell command

bash

```bash
npm install
npm run lint
npm run build
```

Sometimes a single piece of `inline code` is all you need.

03. Important information, without the noise

Not every sentence has the same purpose. Callouts provide useful emphasis without turning a journal into a dashboard of notifications.

> **Note: A quick note**
>
> Simpyll uses MDX, combining Markdown writing with reusable React components.

> **Tip: A useful shortcut**
>
> When documenting a programming problem, include the exact error message, the cause, and the smallest working solution.

> **Information: Behind the scenes**
>
> A centralized MDX component mapping gives code blocks, links, tables, and callouts a consistent visual language.

> **Warning: Before you run that command**
>
> Understand a terminal command before executing it, especially if it modifies files or downloads executable code.

> **Important: The golden rule**
>
> A polished interface should never come at the expense of accessibility, performance, or clarity.

04. Tables that work on your phone

Technical writing loves comparisons. Wide tables, however, can become unreadable on mobile.

Approach	Flexibility	Maintenance	Best for
Static HTML/CSS	High	Low	Simple websites
React + Vite	Very high	Moderate	Interactive experiences
Next.js	Very high	Moderate–high	Full-stack React apps
Astro	High	Low–moderate	Content-focused websites
WordPress	Moderate–high	Moderate	CMS publishing

Approach	Flexibility	Maintenance	Best for
GitHub Pages	Hosting only	Low	Static deployments

> **Tip: Try this on mobile**
>
> Swipe horizontally across the table. The table should scroll while the article stays within the screen width.

05. Images should explain something

A good diagram can make a complicated system easier to understand than several paragraphs of prose.



The Simpyll publishing pipeline: from an idea in Notion to a statically prerendered technical article.

A figure has a caption, meaningful alternative text, and dimensions that help the browser reserve space before the image loads.

06. Complexity, on demand

Some readers want the summary. Others want every implementation detail. A disclosure lets both readers get what they need.

What actually happens when React hydrates a prerendered page?

A prerendered React application delivers HTML containing the initial page content. Hydration attaches client-side React behavior to that existing markup.

The initial client render must agree with the server-rendered output. That matters for theme preferences, browser-only APIs, dynamic measurements, and animations.

Why does Simpyll use static prerendering?

Prerendering includes article content in the HTML delivered to visitors. Content is available before JavaScript runs, crawlers can read it, and GitHub Pages can host the output as static files.

Interactive features can then enhance the existing document.

No unnecessary modal. Just details when you want them.

07. Structure makes ideas easier to follow

A technical tutorial can have a simple sequence:

1. Define the problem.
2. Explain the constraints.
3. Compare possible solutions.
4. Implement the chosen approach.
5. Test the result.
6. Document what you learned.

And useful nested detail:

- **Implementation**
  - Establish a baseline.
  - Make a focused change.
  - Verify the behavior.
- **Validation**
  - Run automated tests.
  - Check mobile layouts.

    - Confirm accessibility.
- **Documentation**
    - Explain the trade-offs.
    - Record unresolved questions.

> The best documentation is not the longest. It is the documentation that helps someone make the next correct decision.

08. Links should behave like links

Visit the Simpyll blog, jump to another section using a heading anchor, or explore MDN Web Docs (https://developer.mozilla.org/).

Links should be recognizable, keyboard-accessible, and predictable—not surprising new tabs or unsafe redirects.

09. References without breaking the flow

Technical claims should be verifiable. React documents how hydration connects a client application to server-rendered markup.[1] Accessibility guidance helps evaluate interactive components.[2]

Footnotes keep the narrative readable while preserving useful references.

10. A safer way to share interactive work

Arbitrary third-party scripts can create security, privacy, and performance concerns. Simpyll's embed component deliberately provides a safe fallback link instead of injecting untrusted scripts.

> Explore the Simpyll source code (https://github.com/simpyll/simpyll)
>
> Open the original resource to view this content.

Less spectacle, more predictable behavior.

11. Reading is an interaction, too

Try a few things while you're here:

- Rotate your phone or resize the browser.

- Follow a heading anchor and use Back.
- Switch between light and dark themes.
- Open a disclosure using the keyboard.
- Copy a code example.
- Scroll across the comparison table.

Good interactions feel natural enough that you barely notice them. If you prefer reduced motion, the content should be just as understandable without decorative animation.

12. Built for people who build things

Simpyll started as a place to document learning, experiments, and projects. I want it to become somewhere technical ideas are explained clearly, experiments are documented openly, and software engineering can intersect with healthcare.

> ⓘ **Important: The real point**
>
> The most immersive part of a technical article is not its animation or typography. It is the moment something complicated finally makes sense. Everything else should support that moment.

Built with React, TypeScript, Vite, Tailwind CSS, MDX, and an unreasonable amount of attention to the little things.

1. React. hydrateRoot reference (https://react.dev/reference/react-dom/client/hydrateRoot). ↩
2. W3C. Web Content Accessibility Guidelines 2.2 (https://www.w3.org/TR/WCAG22/). ↩